

LAB ASSIGNMENT 4

Problem Statement 1 : Parallel Array Summation via Task Decomposition

Objective: Design and implement a parallel C program using **OpenMP** to calculate the total sum of an integer array of size 600. The implementation must demonstrate the use of **task-based parallelism** rather than simple loop scheduling.

Requirements:

1. **Decomposition:** Divide the array into fixed-sized "chunks" (e.g., a STEP_SIZE of 100).
2. **Task Generation:** Use an OpenMP parallel region to iterate through the array. For each chunk, a thread must generate a **task** (`#pragma omp task`) responsible for summing that specific sub-range.
3. **Concurrency:** Multiple threads should participate in both the creation of tasks and the execution of those tasks.
 - Each task must calculate a local partial sum (psum) to minimize data contention.
4. **Check for Synchronisation through omp critical**
 - The final global sum must be updated by each task.
 - You must implement a synchronization mechanism (e.g., `#pragma omp critical`) to prevent race conditions when tasks update the shared global sum variable.
5. You have to print status messages from both the generator threads and the task-executing threads to verify which thread is handling which range of the array.

Problem Statement 2 : Analysing the variable access using different clause

Design and implement a C program using the **OpenMP Tasking API** to investigate how different data-sharing attribute clauses (shared, private, and firstprivate) affect variable persistence and visibility between a parent thread and its generated asynchronous tasks.

Problem Statement 3: Analyzing Task Creation and Execution in OpenMP

Objective: To demonstrate the distinction between **task generation** and **task execution** in a parallel environment.

Requirements:

1. Initialize a parallel region with a specific number of threads.

2. Utilize a single thread (the "producer") to iterate through a loop.
3. Inside the loop, capture the current loop index and the ID of the thread creating the task.
4. Define an OpenMP task that prints "Hello World," the loop index, and the ID of the thread actually *executing* the task.
5. Observe if the thread that **creates** the task is always the one that **executes** it.

Problem Statement 4:

Objective: Implement a parallel program that traverses a singly linked list and processes each node's data using **OpenMP Tasks**.

Implementation Requirements

1. The Producer-Consumer Pattern:

- Use a `#pragma omp single` directive to designate one thread as the **Producer** (the Manager).
- The Manager thread iterates through the list using a while loop.
- For every node encountered, the Manager must generate a **Task** using `#pragma omp task`.

2. Data Scoping:

- Ensure each task receives its own pointer to its specific node. Use the `firstprivate` clause to capture the current value of the node pointer (`temp`) at the moment the task is created.

3. The Consumer (Workers):

- The remaining threads in the parallel region act as **Consumers**. They pick up tasks from the internal OpenMP task queue as soon as they become available.